

GoalWise

Goal-oriented budgeting that tells you what's safe to spend — this week.

MSCS 2101 Software Engineering • Group 3

TEAM 3



Vishal

Project Manager / Leader



Ashutosh

UI / UX Designer



Nati

Assistant PM • Software Engineer / Programmer



Thanh

Quality Assurance & Documentation

THE PROBLEM AND **OUR SOLUTION**

Weekly guesswork vs. one honest number

THE PROBLEM

Budgeting apps organize around spending categories and last month's report card.

Someone saving for one near-term goal is left doing manual math: "What can I actually spend this week without missing my goal?"

Worse, uncertain or unconfirmed income is often counted at face value — producing false confidence.

OUR SOLUTION

GoalWise turns cash, confirmed income, planned expenses, a reserve buffer, and time-to-goal into one **conservative weekly** Safe-to-Spend amount.

It recalculates automatically whenever the inputs change, and it always explains why the number moved.

Deterministic, versioned, and testable — never a black box.

Current Cash

+ Confirmed Income

– Planned Expenses

– Reserve Buffer

➡ Weekly Safe-to-Spend

VISION & COMPETITIVE ANALYSIS

VISION

Empower users to achieve financial goals through goal-driven budgeting.

Success looks like:

- Meet savings goals consistently
- Automatic weekly safe-to-spend updates
- Real-time goal progress
- Less manual budgeting

Feature	GoalWise	YNAB	Monarch
Goal-based budgeting	✓	✓	✓
Weekly recalculation	✓ Auto	Manual	Limited
Forward-looking	✓	Partial	Partial
Recurring expenses	✓	✓	✓
Goal progress	Real-time	✓	✓

USER FLOW

Start



Enter Income & Recurring Expenses



Create Savings Goal



Calculate Safe-to-Spend



Record/Upload Spending



Recalculate Weekly Pace



Display Updated Progress

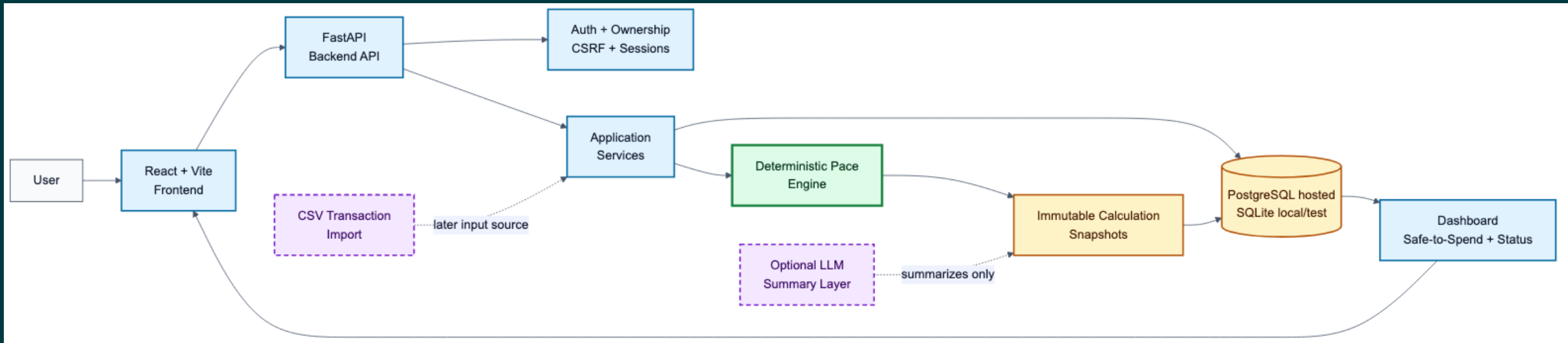


End

TECHNICAL

APPROACH

SYSTEM ARCHITECTURE



DECISIONS

AI Boundary

- AI may summarize deterministic results later
- AI may assist transaction classification later
- AI never calculates or overrides safe-to-spend
- Pace engine remains deterministic and testable

Technical Stack

- React + Vite frontend
- FastAPI backend with REST /api/v1
- SQLAlchemy + Alembic persistence
- PostgreSQL on Railway
- Argon2id auth, HTTP-only sessions, CSRF
- Mermaid ADR/spec documentation

ADR-0001

ADR-0001: Use a Layered Modular Monolith

- **Status:** Accepted
- **Date:** 2026-08-01
- **Deciders:** Nati Seifu
- **Related requirements:** FR-AUTH-001 to FR-AUTH-005, FR-GOAL-001 to FR-GOAL-005, FR-FIN-001 to FR-FIN-007, FR-PACE-001, FR-PACE-006, FR-UI-001 to FR-UI-004, NFR-SEC-005, NFR-MNT-001, NFR-MNT-002

Context

GoalWise needs to deliver an MVP planning loop: authenticate, enter one goal and manual financial assumptions, calculate a deterministic weekly safe-to-spend amount, persist a calculation snapshot, and show a dashboard.

The main architectural drivers are deliverability, determinism, privacy, explainability, and changeability. The system should be small enough to build quickly, but structured enough that post-MVP features such as CSV import, weekly automation, and optional AI summaries can be added without rewriting the financial core.

Decision

We will use a layered modular monolith:

- FastAPI routers for HTTP endpoints under `/api/v1`.
- Pydantic schemas for request and response validation.
- Application services for business rules, recalculation triggers, ownership checks, and snapshot creation.
- Repositories for SQLAlchemy query isolation.
- SQLAlchemy models and Alembic migrations for persistence.
- A pure `pace_engine` module with no web, database, session, frontend, or AI dependencies.

Alternatives considered

- **Layered modular monolith** - Chosen because it best fits MVP speed, simple deployment, local testability, and clear boundaries. It requires discipline to avoid service and repository leakage.
- **Client-heavy application** - Rejected because financial logic would be easier to tamper with, harder to test centrally, and harder to audit.
- **Microservices** - Rejected because operational complexity and distributed data consistency are unjustified for one product loop.
- **Event-driven architecture** - Rejected because asynchronous projections are unnecessary for the MVP dashboard and would slow delivery.
- **AI-agentic architecture** - Rejected because it is a poor fit for deterministic financial calculation and review defensibility.

Consequences

Positive:

- The MVP can run as one backend service and one frontend.
- The pace engine can be tested with pure golden tests.
- User ownership and validation rules can be enforced centrally.
- Future adapters can be added around the core instead of inside it.

Negative:

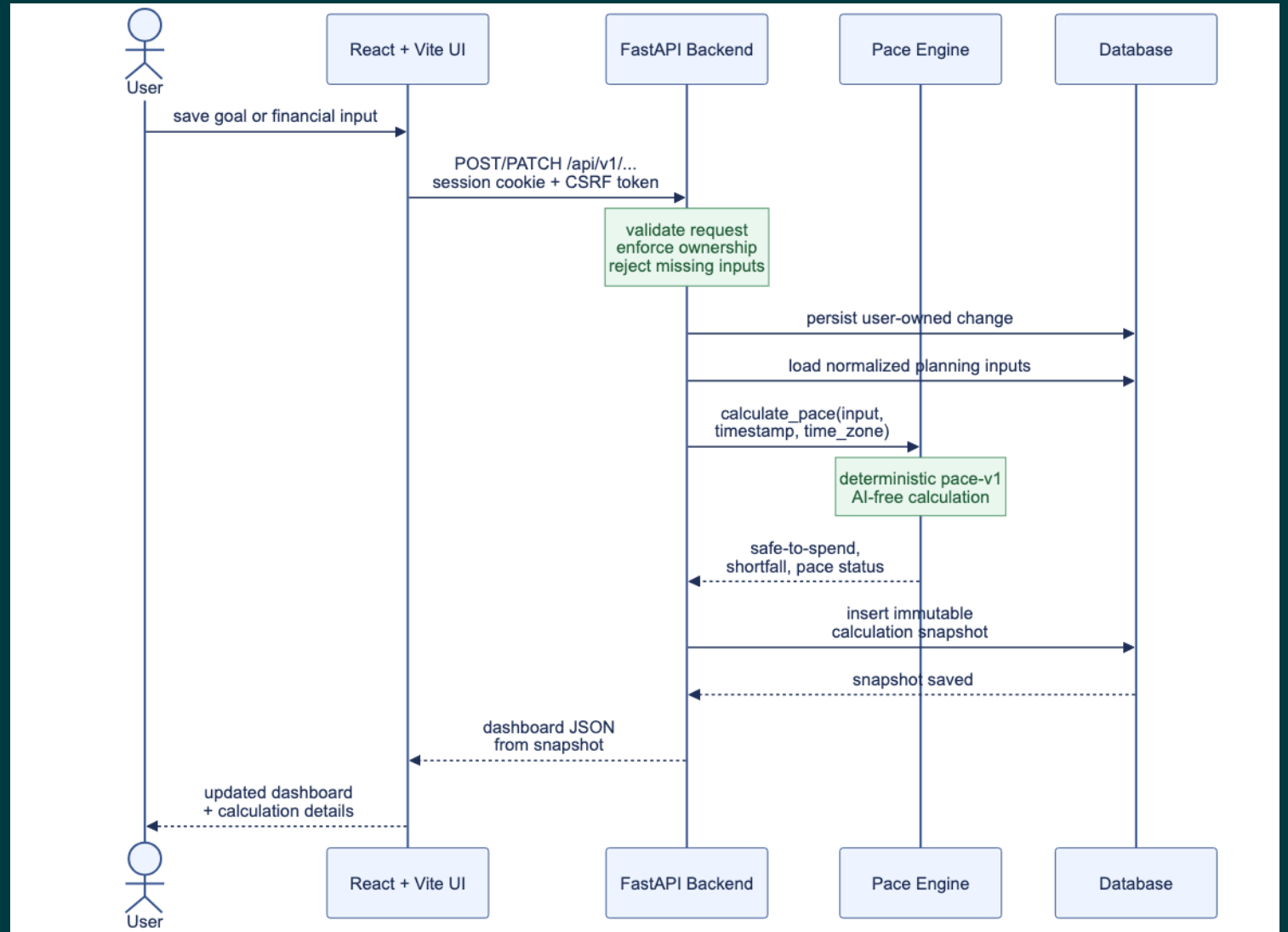
- The codebase must maintain clear module boundaries manually.
- If future traffic or team ownership grows, modules may need to be extracted later.

Neutral / follow-ups:

- Routers must not contain financial calculation logic.
- The pace engine must not import FastAPI, SQLAlchemy, session, frontend, or AI modules.
- Service tests must cover ownership checks and recalculation behavior.

SEQUENCE DIAGRAM

"Set a financial goal"



CHALLENGES, RISKS **AND NEXT STEPS**

PRACTICAL CHALLENGES

UNREALISTIC EXPECTATION: Overly Optimistic Safe-to-Spend Calculation

- Income:
 - o Overestimates income
 - o Relies on uncertain income in the total income calculation (freelance projects)
- Expense:
 - o Underestimate expenses
 - o Lack of understanding about unpredictable expenses (medical bills, urgent household equipment repairs, car tune-up, etc)

USER INPUT INCORRECT DATA & LOW ENGAGEMENT

- Missing incomes/expenses, incorrect amounts & not entering enough data

PRIVACY AND SECURITY BREACHES

- Personal finance data (volume and category)
- Financial behavior exposure (habit of spending, invest/withdraw routine,...)

SOFTWARE CALCULATION BUGS

MITIGATION STRATEGY (1)

UNREALISTIC EXPECTATION: Overly Optimistic Safe-to-Spend Calculation

- Rely heavily on predictable income and underestimate uncertain income (commission, bonuses, freelance,...)
- Use % conservative reserve buffer (set safety margin)
- Display how recommendations are made (saving analysis)
- Real-time update (or weekly update) per new entries

NEXT STEPS:

- Test against predefined financial scenarios
- User sets the safety margin according to average %
- Reduce the use of AI in recommendation; focus on if-then rules
- Weekly goals notice feature

MITIGATION STRATEGY (2)

USER INPUT INCORRECT DATA & LOW ENGAGEMENT

- Verify steps after data entry (review)
- Reuse past data for auto-fill (during entry) & verification (after entry)

NEXT STEPS:

- Validation feature from the user (feedback) for important requirements

MITIGATION STRATEGY (3)

PRIVACY AND SECURITY BREACHES

- User authentication is required for entry/edit, view, and extract data/report/recommendation from software
- User confirmation for any suggestion/recommendation from software
- No AI/automation besides suggestion/financial plan assumption

NEXT STEPS:

- User information allowance policy
- Two-step authentication sign-in
- Password rules

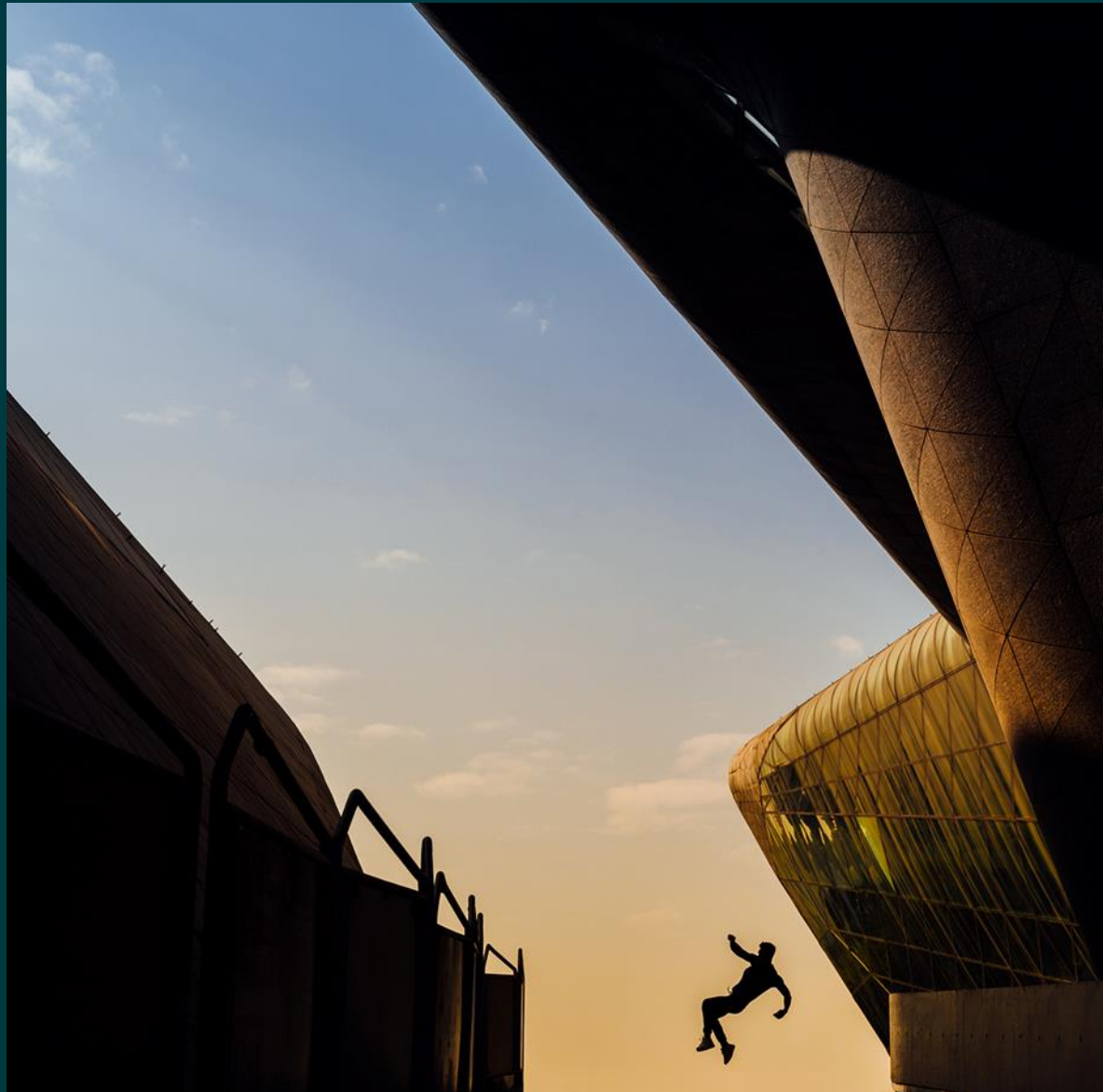
MITIGATION STRATEGY (4)

SOFTWARE CALCULATION BUGS

- Code review & automated cross-test (using multiple AI)
- Adjust predefined financial scenario testing

NEXT STEPS:

- AI agent selection criteria
- Build predefined financial scenario testing



Q&A

Panel discussion session